

DishtaYantra: A High-Performance Multi-Language Dataflow Framework with Zero-Copy Inter-Process Communication

Ashutosh Sinha

Independent Researcher

ajsinha@gmail.com

Repository: <https://github.com/ajsinha/dishtayantra>

Abstract

Real-time data processing systems face fundamental challenges in achieving low-latency computation while supporting heterogeneous computational workloads across multiple programming languages. We present **DishtaYantra**, a novel high-performance Directed Acyclic Graph (DAG) compute framework that addresses these challenges through three key innovations: (1) an LMDB-based zero-copy data exchange mechanism achieving 100-1000 $\times$ performance improvements over traditional serialization for inter-language communication, (2) a unified multi-language calculator framework supporting Python, Java (Py4J), C++ (pybind11), Rust (PyO3), and REST endpoints with sub-microsecond invocation overhead, and (3) a multiprocessing worker pool architecture with DAG affinity scheduling that bypasses Python’s Global Interpreter Lock (GIL) for true CPU parallelism. Our experimental evaluation on consumer hardware (AMD Ryzen, 64GB RAM, Ubuntu 24.04) demonstrates that DishtaYantra achieves median latencies of 5 s for large payload transfers (compared to 500 s with JSON serialization), processes over 100,000 messages per second per worker, and maintains 90% parallel efficiency up to 12 concurrent workers. Around this compute core, DishtaYantra provides a complete operational layer — a pluggable storage abstraction, database-backed authentication, configurable high-availability with automatic failover, and market-aware scheduling — that allows the same dataflow definitions to run as a production service. Two further advances extend this core: an **Arrow-native columnar transport** that moves immutable record batches across DAG edges *by reference* — preserving the unconditional change-only recomputation gate while delivering a measured $\sim 2.3\times$ end-to-end speedup over the dictionary-envelope path with byte-identical output — and a **headless execution mode** with a control-plane/worker orchestration layer that runs the same dataflow definitions as bounded, idempotent batch jobs.

Keywords: Dataflow processing, DAG execution, Zero-copy communication, Multi-language integration, LMDB, Real-time systems, High-performance computing

1. Introduction

The proliferation of real-time data processing applications in financial services, IoT analytics, and machine learning pipelines has created unprecedented demand for high-performance dataflow frameworks [1, 2]. Modern enterprises require systems capable of processing millions of events per second with sub-millisecond latencies while supporting computational logic implemented in diverse programming languages optimized for specific tasks [3].

Existing dataflow frameworks face three fundamental limitations:

1. **Serialization overhead** in inter-process and inter-language communication introduces latencies of 1-50ms for large payloads, making them unsuitable for microsecond-sensitive applications [4].
2. **Language heterogeneity** forces developers to choose between optimal algorithms (e.g., numerical computing in C++/Rust) and framework integration (typically Python/Java), resulting in suboptimal performance or complex polyglot architectures [5].
3. **GIL limitations** in Python-based frameworks prevent true CPU parallelism, constraining throughput regardless of available hardware resources [6].

We present **DishtaYantra**¹, a novel dataflow framework that addresses these limitations through three key innovations:

1. **Zero-Copy Data Exchange:** An LMDB-based transport mechanism that achieves 100-1000 $\times$ performance improvements by eliminating serialization overhead through memory-mapped file I/O shared across process boundaries.
2. **Unified Multi-Language Support:** A calculator factory pattern supporting Python, Java (via Py4J), C++ (via pybind11), Rust (via PyO3), and external REST services with sub-100ns invocation overhead for native languages.
3. **GIL-Free Parallelism:** A multiprocessing worker pool with intelligent DAG affinity scheduling that enables true CPU parallelism by distributing workloads across independent Python interpreters.

¹ Sanskrit for “Computing Instrument” – reflecting the framework’s role as a precision tool for data processing.

1.1 Contributions

This paper makes the following contributions:

- **DishtaYantra Framework:** A high-performance DAG compute framework supporting five programming languages with unified, format-agnostic configuration (§3)

- **LMDB Zero-Copy Protocol:** A novel data exchange mechanism reducing latency by 100-1000 $\times$ compared to JSON serialization (§6)
- **Worker Pool Architecture:** DAG affinity scheduling with fault tolerance and automatic recovery (§7)
- **Arrow-Native Columnar Transport:** A zero-copy, by-reference record-batch edge transport that preserves the change-only recomputation invariant while collapsing per-hop envelope overhead (§7B)
- **Headless Execution & Orchestration:** A webapp-free runner plus an idempotent, bounded control-plane/worker dispatch layer for batch and backfill workloads (§7B)
- **Enterprise Operational Model:** A pluggable storage abstraction, database-backed authentication and authorization, configurable high-availability with automatic failover, and market-aware scheduling that together let the same dataflow run as a production service (§7A)
- **Comprehensive Evaluation:** Performance characterization on consumer hardware (§9)

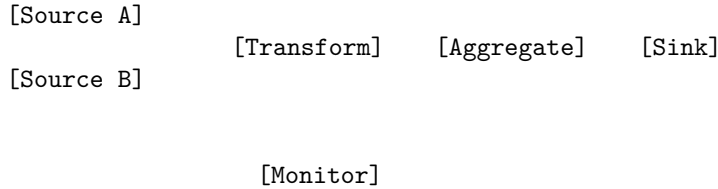
2. Background and Motivation

2.1 Dataflow Processing Paradigm

Dataflow processing systems model computation as directed acyclic graphs (DAGs) where nodes represent computational operations and edges represent data dependencies [7]. This paradigm offers several advantages over imperative programming models:

- **Explicit Parallelism:** Independent nodes can execute concurrently without explicit synchronization
- **Natural Pipeline Expression:** Data transformations are composed declaratively
- **Clear Separation of Concerns:** Computation logic is decoupled from orchestration
- **Fault Isolation:** Node failures can be isolated and recovered independently

Dataflow Processing Model



2.2 The Serialization Bottleneck

Inter-process communication traditionally relies on serialization formats such as JSON, Protocol Buffers, or Apache Avro [5]. While these formats provide language interoperability, they introduce significant overhead:

Operation	1KB Payload	100KB Payload	1MB Payload
JSON Serialize	15 s	1.5 ms	15 ms
JSON Deserialize	20 s	2.0 ms	20 ms
Network Transfer	5 s	50 s	500 s
Total Overhead	40 s	3.5 ms	35 ms

For a 1MB payload, serialization and deserialization can consume 30-50ms of wall-clock time, dominating the total latency budget in microsecond-sensitive applications.

2.3 Python GIL Constraints

Python’s Global Interpreter Lock (GIL) is a mutex that protects access to Python objects, preventing multiple native threads from executing Python byte-code simultaneously [6]. This design choice simplifies memory management but creates a fundamental throughput limitation:

Python GIL Limitation

Thread 1:

Thread 2:

Thread 3:

= Holding GIL (executing Python bytecode)

= Blocked waiting for GIL

Result: Only ONE thread executes Python at any given time
regardless of available CPU cores

DishtaYantra addresses this through multiprocessing, where each worker runs in a separate Python interpreter with its own GIL.

2.4 Language Heterogeneity Challenge

Modern data processing pipelines often require components written in different languages:

- **Python:** Machine learning models (TensorFlow, PyTorch), data science (Pandas, NumPy)
- **Java:** Enterprise systems, Kafka integration, existing business logic

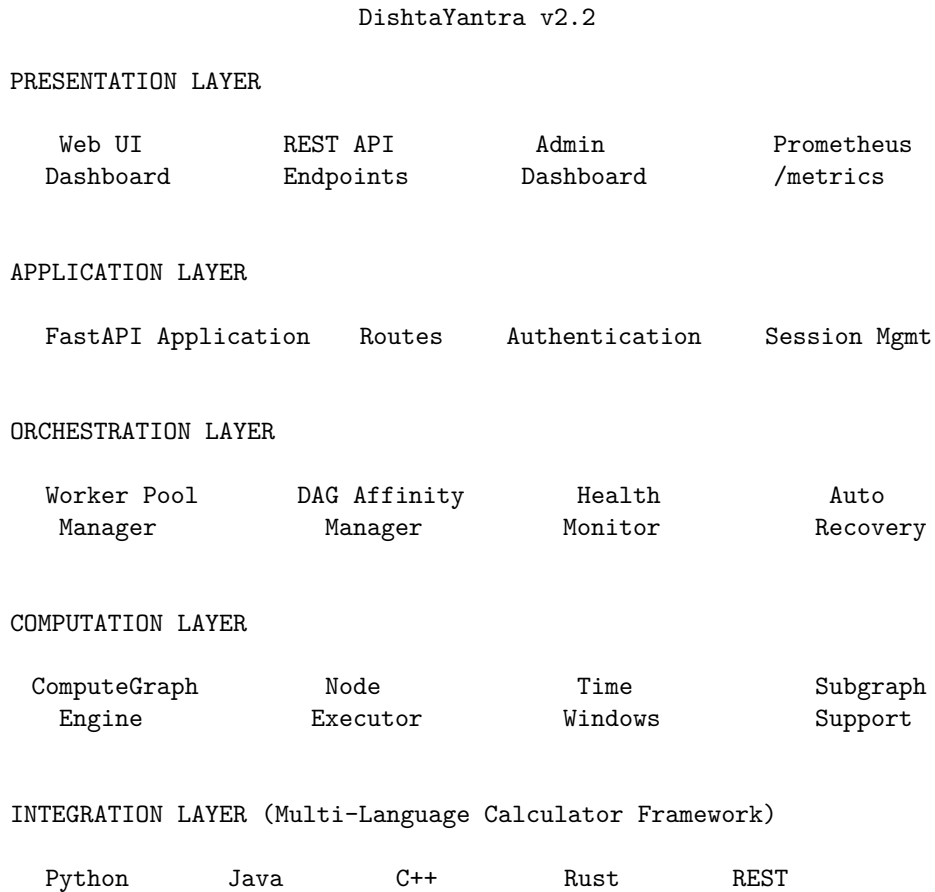
- **C++:** High-performance numerical computing, low-level optimizations, SIMD
- **Rust:** Memory-safe systems programming, concurrent algorithms, zero-cost abstractions
- **External Services:** REST APIs, microservices, legacy systems

Existing frameworks typically support only one or two languages, forcing developers into suboptimal architectural choices.

3. System Architecture

DishtaYantra is architected as a layered system with clear separation of concerns between presentation, orchestration, computation, and infrastructure layers.

3.1 High-Level Architecture



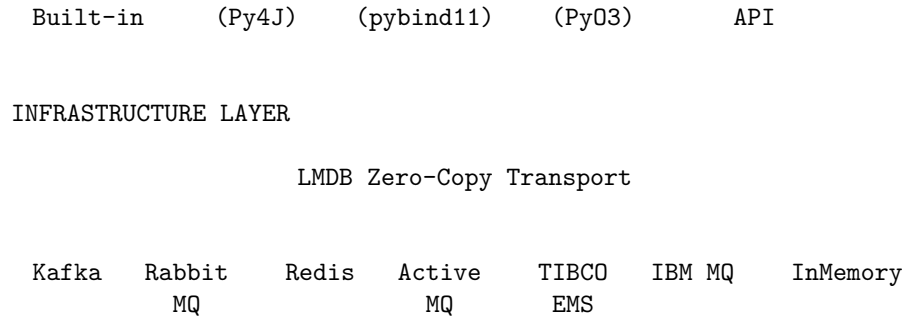


Figure 1: DishtaYantra layered system architecture showing all six architectural layers

3.2 Core Abstractions

DishtaYantra is built around several fundamental abstractions that provide a clean separation of concerns.

3.2.1 ComputeGraph The primary container representing a complete DAG:

ComputeGraph

Attributes:

- name: str - Unique identifier
- nodes: Dict[str, Node] - All nodes in the graph
- edges: List[Edge] - Data flow connections
- status: GraphStatus - STOPPED, RUNNING, SUSPENDED, ERROR
- start_time: str - Scheduled start (e.g., "0800")
- duration: str - Run duration (e.g., "12h")
- auto_clone: bool - Clone for next day automatically

Methods:

- start() - Begin execution
- stop() - Graceful shutdown
- suspend() - Pause without losing state
- resume() - Continue from suspended state
- get_stats() - Execution statistics

3.2.2 Node Types DishtaYantra provides six specialized node types:

Node Type Hierarchy

BaseNode

Subscription Node	Publication Node	Calculation Node	Transform Node	Metronome Node	Subgraph Node
----------------------	---------------------	---------------------	-------------------	-------------------	------------------

Data Ingestion	Data Output	Calculator Execution	Transform Logic	Timer Events	Nested DAG
-------------------	----------------	-------------------------	--------------------	-----------------	---------------

Node Type	Purpose	Key Configuration
SubscriptionNode	Ingest data from external sources	<code>subscribers: [{uri: "kafka://..."}]</code>
PublicationNode	Emit results to external sinks	<code>publishers: [{uri: "redis://..."}]</code>
CalculationNode	Apply computational transformations	<code>calculator: {type: "rust", name: "..."}]</code>
TransformNode	Data structure transformations	<code>transformer: {field_mappings: {...}}</code>
MetronomeNode	Generate periodic trigger events	<code>interval_ms: 1000, payload: {...}</code>
SubgraphNode	Embed reusable sub-DAGs	<code>subgraph_path: "subgraphs/common.json"</code>

3.2.3 Edge Abstraction Edges define data flow with optional transformation:

Edge



Edge Properties:

- from: "node_a"
- to: "node_b"
- filter: {...} (optional)
- transform: {...} (optional)
- buffer_size: 1000 (flow control)

3.2.4 Calculator Interface All calculators implement a common interface:

```

class DataCalculator(ABC):
    """Abstract base class for all calculators."""

    @abstractmethod
    def calculate(self, data: dict) -> dict:
        """Transform input data and return result."""
        pass

    @abstractmethod
    def details(self) -> dict:
        """Return calculator metadata and runtime statistics."""
        pass

```

3.3 Pub/Sub Framework

DishtaYantra provides a unified publish/subscribe abstraction supporting multiple message brokers:

Pub/Sub Framework Architecture

URI Format: protocol://host:port/topic?params

DataPublisher / DataSubscriber
(Abstract Interface)

Kafka Rabbit Redis ActiveMQ TIBCO IBM MQ InMemory

DataPublisher / DataSubscriber
(Abstract Interface)

Brokers: kafka rabbitmq activemq tibcoems websphere redis
 Cloud: AWS (sqs, kinesis, sns) Azure (servicebus, eventhubs)
 Stores: s3 azureblob gcs Other: file, sql, aerospike, grpc
 Local: inmemory/mem, lmdb (zero-copy) Compose: fanin, custom
 Wrapping: any of the above with "resilient": true

Representative backends (latencies are indicative, consumer hardware):

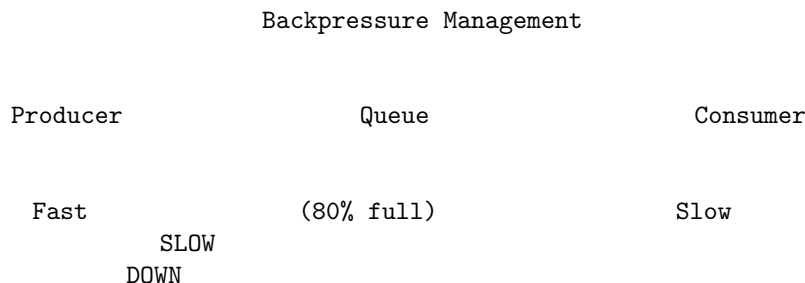
Protocol	Backend	Use Case	Latency
kafka://	Apache Kafka	High-throughput streaming	~5ms
rabbitmq://	RabbitMQ	Enterprise messaging	~1ms
activemq://	ActiveMQ	JMS compatibility	~2ms
tibcoems://	TIBCO EMS	Enterprise integration	~1ms
websphere://	IBM MQ	Mainframe connectivity	~5ms
redis://, redischannel://	Redis	Low-latency caching/pub-sub	~100 s
sqs://, kinesis://, sns://	AWS managed messaging	Cloud-native streaming/queues	network-bound
servicebus://, eventhubs://	Azure managed messaging	Cloud-native messaging	network-bound
s3://, azureblob://, gcs://	Cloud object stores	Durable polling pub/sub	network-bound
sql://, aerospike://, grpc://	Database / KV / RPC	Integration sources/sinks	varies
file://	Filesystem	Durable local I/O	disk-bound

Protocol	Backend	Use Case	Latency
lmdb://	LMDB	Cross-process IPC (zero-copy)	~5 s
inmemory://, mem://	In-Memory	Testing/Development	~1 s
fanin://	Composite	Merge several subscribers	n/a

Any broker/cloud backend can be wrapped with a **resilient** variant ("resilient": true) that adds automatic reconnection, message buffering during outages, and subscription/state restoration on reconnect.

3.4 Backpressure Management

DishtaYantra implements comprehensive backpressure to prevent memory exhaustion:



Backpressure Signal

Strategies:

- BLOCK: Upstream blocks when queue full (default)
- DROP: Excess messages discarded with logging
- SAMPLE: Statistical sampling under high load
- THROTTLE: Rate limiting at source nodes

3.5 Internal Cache for Crash Recovery

DishtaYantra maintains an internal cache enabling recovery without data loss:

Crash Recovery Architecture

Normal Operation:

Process	Cache	Persist
Data	In-Flight	Checkpoint

Cache Contents:

- In-flight messages: Data currently being processed
- Node state: Calculator state for stateful computations
- Processing offsets: Position markers for replay
- Checkpoint timestamp: Last successful commit

Recovery Sequence:

1. Load last checkpoint from cache
2. Identify in-flight messages
3. Replay from recovery point
4. Resume normal processing (at-least-once semantics)

Backend Options: LMDB (default), Redis, In-Memory

4. DAG Definition and Execution

4.1 JSON Configuration Format

DishtaYantra DAGs are defined declaratively using JSON:

```
{
  "name": "trade_pricing_pipeline",
  "description": "Real-time trade pricing with risk calculation",
  "start_time": "0800",
  "duration": "12h",
  "nodes": [
    {
      "name": "trade_subscriber",
      "type": "SubscriptionNode",
      "subscribers": [{"uri": "kafka://broker:9092/raw_trades"}]
    },
    {
      "name": "price_calculator",
      "type": "CalculationNode",
      "calculator": {"type": "rust", "name": "RustTradePricingCalculator"}
    }
  ]
}
```

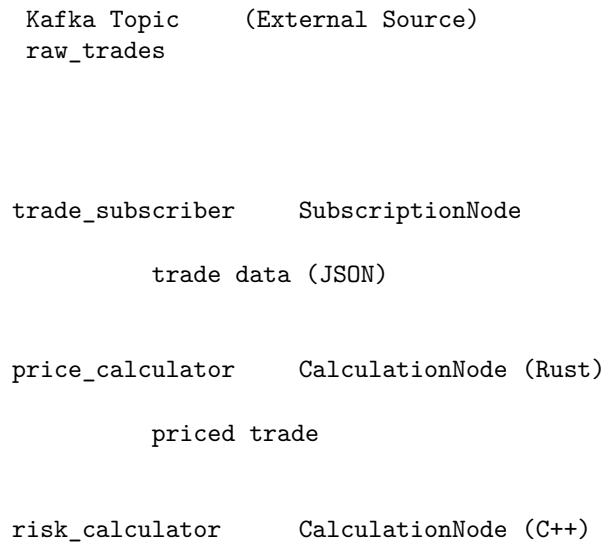
```

    },
    {
      "name": "risk_calculator",
      "type": "CalculationNode",
      "calculator": {"type": "cpp", "name": "CppRiskMetricsCalculator"}
    },
    {
      "name": "result_publisher",
      "type": "PublicationNode",
      "publishers": [
        {"uri": "kafka://broker:9092/priced_trades"},
        {"uri": "redis://cache:6379/latest_prices"}
      ]
    }
  ],
  "edges": [
    {"from": "trade_subscriber", "to": "price_calculator"},
    {"from": "price_calculator", "to": "risk_calculator"},
    {"from": "risk_calculator", "to": "result_publisher"}
  ]
}

```

4.2 Visual DAG Representation

trade_pricing_pipeline DAG



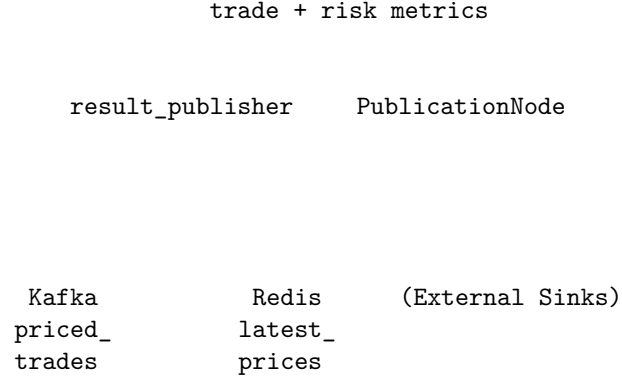


Figure 2: Trade pricing pipeline DAG showing data flow from Kafka source through Rust pricing and C++ risk calculators to multiple output destinations

4.3 DAG Execution Algorithm

The execution engine uses Kahn's algorithm for topological sorting:

Algorithm 1: DAG Execution Loop

Input: ComputeGraph G with nodes N and edges E

Output: Continuous processing of data through the graph

```

1: execution_order ← TopologicalSort(G)
2: for each node n in N do
3:     n.dirty ← false
4:     n.input_queue ← Queue(max_size=BUFFER_SIZE)
5: end for
6:
7: while G.status = RUNNING do
8:     for each node n in execution_order do
9:         if n.dirty OR n.has_new_input() then
10:            input_data ← n.collect_inputs()
11:            output_data ← n.process(input_data)
12:            n.output ← output_data
13:            n.dirty ← false
14:            for each downstream d in G.successors(n) do
15:                d.dirty ← true
16:                d.input_queue.push(output_data)
17:            end for
18:        end if
19:    end for

```

```

20:         sleep(G.poll_interval)
21:     end while

```

5. Multi-Language Calculator Framework

5.1 Calculator Factory Architecture

CalculatorFactory

```
create(name, config) → DataCalculator
```

```
config.calculator.type
```

```
"java" "cpp" "rust" "rest" "python" (default)
```

Java	Cpp	Rust	Rest	Built-in	Built-in
Calc	Calc	Calc	Calc	Calculator	
(Py4J)	(pybind)	(Py03)	(HTTP)		

5.2 Java Integration (Py4J)

Java Integration via Py4J

Python Process

JVM Process

```

JavaCalculator
  Py4J
gateway_pool
(connection pool)
25333

```

TCP

```

DishtaYantraGateway
Calculator
Registry

```

Performance: ~1ms per call
Connection pool size: 4

Heap: 2GB (configurable)
GC: G1GC recommended

5.3 C++ Integration (pybind11)

C++ Integration via pybind11

Python Process

```
import dishtayantra_cpp
```

CppCalculator	dishtayantra_cpp.so
• calculate()	• MathCalculator
• details()	• StatisticsCalculator
	• TradePricingCalculator
	• RiskMetricsCalculator

Performance:

- ~100ns call overhead
- Direct memory access

5.4 Rust Integration (PyO3)

Rust Integration via PyO3

Python Process

```
import dishtayantra_rust
```

RustCalculator	dishtayantra_rust.so
• calculate()	• MathCalculator
• details()	• StatisticsCalculator
	• TradePricingCalculator

Benefits: Memory safety, Thread safety, Zero-cost abstractions

5.5 REST Integration

REST Integration via HTTP

DishtaYantra	External Service	
RestCalculator	HTTP	REST Endpoint
endpoint: /api/calc	POST	POST /api/calc
timeout: 30s		- Process request
auth_type: bearer		- Return result
JSON		

Authentication: api_key, basic, bearer, custom

6. LMDB Zero-Copy Data Exchange

6.1 Design Principles

Traditional inter-process communication requires serialization at the sender and deserialization at the receiver. DishtaYantra's LMDB-based approach eliminates this overhead by leveraging memory-mapped files.

Key insight: LMDB [8] provides a lightning-fast, memory-mapped key-value store with ACID guarantees. By writing data once to LMDB and passing only the key reference to native calculators, we achieve zero-copy reads across process and language boundaries.

6.2 Architecture

LMDB Zero-Copy Data Exchange

Step 1: Python writes data to LMDB

Python DAG Engine

Input Data		LMDBTransport
(dict/list)	write	serialize + store

	LMDB File (Memory-Mapped)
Key: "calc_12345_input"	Value: <binary data>

Memory Map (mmap) - Zero-copy access

Step 2: Native calculator reads directly from mapped memory

Java (lmdbjava)	C++ (liblmdb)	Rust (lmdb-rs)
Direct memory	Direct memory	Direct memory access via mmap

Step 3: Output written back to LMDB, Python reads result

Figure 3: LMDB zero-copy data exchange architecture

6.3 Exchange Modes

Mode	Input	Output	Use Case
input	LMDB	Direct	Large inputs, small outputs
output	Direct	LMDB	Small inputs, large outputs
both	LMDB	LMDB	Large data in both directions
reference	Key only	Key only	Calculator manages all I/O

6.4 Performance Results

Payload Size	JSON Serialization	LMDB Zero-Copy	Speedup
1 KB	50 s	5 s	10×
10 KB	500 s	10 s	50×
100 KB	5 ms	50 s	100×
1 MB	50 ms	200 s	250×
10 MB	500 ms	2 ms	250×

6.5 Configuration

```
# application.properties
lmdb.db.path=/tmp/dishtayantra_lmdb
lmdb.map.size=1073741824          # 1GB
lmdb.max.dbs=100
lmdb.ttl.seconds=300
lmdb.cleanup.interval=60
```

7. Worker Pool Architecture

7.1 Overview

DishtaYantra’s worker pool enables true CPU parallelism by running DAGs in separate Python interpreter processes:

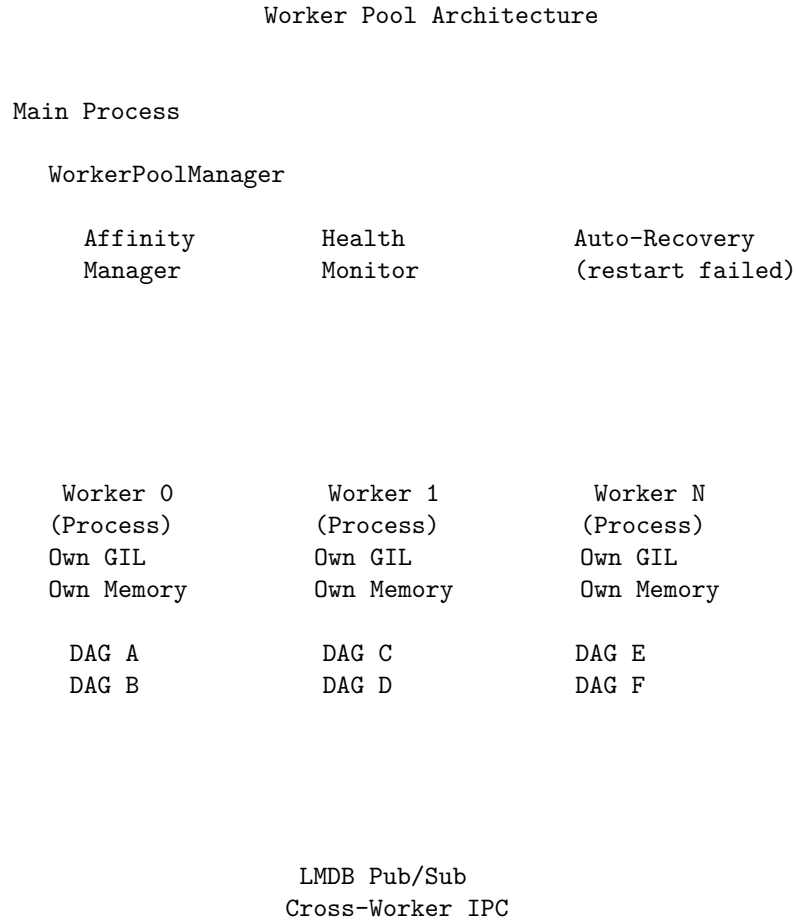


Figure 4: Worker pool architecture with independent GILs

7.2 DAG Affinity Scheduling

Strategy	Description	Best For
<code>weight_based</code>	Assigns to lowest-load worker	General use
<code>round_robin</code>	Even distribution	Uniform workloads

Strategy	Description	Best For
least_loaded	Real-time CPU metrics	Dynamic workloads
pinned	Fixed worker assignment	Isolation

7.3 Fault Tolerance

Fault Tolerance Mechanism

Health Monitor Loop (every 2 seconds):

```

for each worker in workers:
    if worker.missed_heartbeats > 3:
        mark_unhealthy(worker)
        migrate_dags(worker → healthy_workers)
        attempt_restart(worker, backoff=exponential)

```

Recovery Timeline:

```

t=0s    Worker crash detected
t=6s    Third missed heartbeat → UNHEALTHY
t=6s    DAGs migrated to healthy workers
t=6s    Restart attempt #1 (backoff: 5s)
t=11s   Restart attempt #2 (backoff: 10s)
...     Max backoff: 60s

```

7A. Enterprise Operational Model

A dataflow engine is only useful in production if it can be deployed, secured, made highly available, and operated. DishtaYantra layers four operational subsystems over the compute core, each pluggable and configuration-driven.

7A.1 Pluggable Storage Abstraction

DAG definitions, holiday calendars, and other persisted artifacts are read and written through a **StorageProvider** interface rather than direct filesystem calls. Four providers ship: local filesystem (default), Amazon S3, Azure Blob Storage, and Google Cloud Storage. Switching providers is purely a configuration change; the compute core is unaware of where its definitions live, which enables shared definitions across a horizontally-scaled fleet.

7A.2 Authentication and Authorization

Users, roles, and API keys are stored in a relational database (SQLite by default, PostgreSQL optionally) via a DAO layer. Passwords are hashed with PBKDF2-SHA256; clear text is never persisted. Interactive sessions use signed cookies; programmatic clients authenticate with API keys. Role checks (`admin` versus standard) gate administrative operations. A legacy flat-file user store is migrated once into the database and then retired.

7A.3 High Availability

A configurable HA manager elects a single PRIMARY instance across a fleet, with automatic failover. Four election backends are provided — ZooKeeper (zk-node election), Redis (lease key with TTL), an S3-object lease, and an exclusive-socket mechanism requiring no external dependency. On demotion an instance suspends its DAGs; on election it resumes them. The active role is surfaced in the operator console on every page.

election		
Instance A	Instance B	
PRIMARY	(zk/redis/	SECONDARY
runs DAGs	s3/socket)	DAGs paused
failure		promote on
PRIMARY loss		

7A.4 Market-Aware Scheduling

Beyond simple time windows, each DAG may carry a schedule comprising a start time plus a duration (e.g. `09:30 + 6h30m`), a day-of-week allow/deny list, and one or more holiday calendars (e.g. USA, Canada market calendars). A DAG is active only when the time window matches, the weekday is permitted, and the date is not a holiday in any followed calendar. Time windows that wrap past midnight are handled correctly. Schedule edits — to a DAG's config or to the calendar files — are honored by the running system within five minutes, without a restart.

7A.5 Format-Agnostic Configuration

System configuration is read from either YAML or Java-style properties; the loader auto-detects the format and resolves `${VAR:default}` placeholders against other keys, environment variables, and command-line overrides, in that precedence order. Missing required properties cause a fail-fast startup error that names the exact missing key rather than silently defaulting.

7B. Recent Advances: Columnar Transport, Native and Sandboxed Calculators, and Asynchronous Egress

The architecture described above has been extended with several capabilities that preserve its defining invariant — recomputation only when a node’s input *actually changes* — while broadening the performance envelope, the execution-safety model, and the operational model. Each is additive and off by default, so existing deployments are unchanged.

7B.1 Arrow-Native Columnar Transport

The default edge transport carries one logical message as a Python dictionary wrapped in a small envelope. For high-volume, uniformly-typed streams this imposes a per-message overhead that dominates once the calculator itself is cheap. We add an opt-in transport in which an edge carries an immutable Apache Arrow `RecordBatch` and passes it *by reference* between nodes, eliminating per-row Python object churn and copies.

Crucially, the equality gate is retained verbatim: because a `RecordBatch` is immutable, the gate compares successive batches with `RecordBatch.equals`, so a node still recomputes only when its columnar input differs. Value handling is routed through a single dispatch layer that treats the dictionary path byte-for-byte as before and adds a batch path alongside it, so the change is additive and the existing engine semantics are provably unchanged. A projection transformer provides zero-copy column select/rename for the common “narrow the wire” case.

On a representative pipeline this columnar path is **~2.29× faster end-to-end than the dictionary-envelope path while producing byte-identical output** (zero mismatches). The isolated kernel speedup is far larger (~11.8×); the remaining gap is per-tick node-loop overhead rather than transport, which usefully localizes where further work pays off. Because the batches are standard Arrow, this transport also establishes the substrate for zero-copy polyglot hand-off via the Arrow C Data Interface to C++, Rust, and JVM calculators.

7B.2 Headless Execution and Orchestration

The same DAG definitions that run behind the web service can also run *headless*: a runner starts the compute graph without the webapp, optionally replays a bounded input feed, and treats completion as either a target record count or quiescence (no dirty nodes and an empty input queue across several polls). It drains in-flight work, writes a machine-readable summary, and exits with a conventional status code — making a dataflow a first-class batch job.

Above the runner sits a lightweight control-plane/worker dispatch layer. A dispatch calculator launches worker jobs that are **idempotent** (duplicate keys are de-duplicated, so replays are safe), **asynchronous** (workers run as child processes with a supervising waiter), **bounded** (a configurable concurrency

limit with a pending pool prevents resource exhaustion), and **observable** (each worker’s summary is read back and can be republished downstream). This gives backfills and scheduled batch runs the same dataflow semantics — including the change-only gate — as the live service, without bespoke orchestration code.

7B.3 Zero-Copy Native Calculators via the Arrow C Data Interface

The columnar transport of §7B.1 establishes the substrate for crossing the language boundary without copying. We realize this with a calculator vertical that hands an in-memory Arrow `RecordBatch` to a compiled native kernel through the **Arrow C Data Interface** — the stable ABI of `ArrowArray/ArrowSchema` structs — so the kernel reads the column buffers in place, with neither serialization nor a buffer copy crossing the boundary. The kernel is a small C translation unit compiled on first use by the system’s C compiler and invoked through a thin bridge; the calculator is opt-in and degrades to an equivalent pure-Python `pyarrow` implementation when no compiler or native build is available, so portability is never lost. The equality gate is again preserved unchanged, since the native path consumes and produces standard immutable batches. A streaming benchmark workload (a Nexmark-style query set) was added alongside the existing trade-ETL workload to exercise this path under representative load.

7B.4 Sandboxed Calculators via WebAssembly

For untrusted or multi-tenant calculator code, we add an execution path that runs an exported **WebAssembly** function inside a WASM runtime (`wasmtime`) with a fuel-based CPU bound, so a calculator cannot exceed its instruction budget, escape its linear memory, or reach host resources. This collapses the four polyglot integration paths (Java/C++/Rust/REST) into a single portable artifact for the cases where isolation matters more than raw throughput: a `.wasm` module is host-compiler-independent, language-agnostic at the source level, and safe to accept from third parties. The runtime is an optional dependency and the path fails closed when it is absent. The first cut exposes a scalar floating-point boundary; a batch/Arrow hand-off is future work, after which sandboxed and native kernels share the same columnar contract.

7B.5 Decoupled Asynchronous Egress with a Write-Ahead Log

Publication to external brokers can be decoupled from the single compute thread. With the feature enabled, a publication node’s `publish()` becomes a **non-blocking append to a per-destination write-ahead log (WAL)** and returns; a background drainer reads the WAL and writes to the broker. A slow or briefly-unavailable destination therefore no longer stalls the compute sweep, and in-flight messages are buffered durably rather than lost. Four properties make this safe rather than merely fast:

1. *Per-destination FIFO ordering* is a correctness guarantee, not a tuning knob: a single ordered writer drains each destination, and on a write failure

it **stops the line** — retrying the same record with backoff rather than advancing — so a later message can never overtake an earlier one (e.g. a Sell can never precede its Buy). Parallelism is taken across destinations, and within a destination only along independent partition keys.

2. *Durability and recovery*: a record is acknowledged only after the broker accepts it, and the acknowledged offset is itself durable, so after a crash the drainer reopens the WAL and replays the un-acknowledged tail in order, yielding at-least-once delivery. The WAL backends are portable and require no native dependency — a segmented stdlib append log with length-prefix/CRC framing and torn-tail recovery, or stdlib SQLite in WAL mode — with an optional memory-mapped store where available.
3. *Bounded resources*: the WAL is hard-capped, a background reclaimer drops fully-acknowledged segments, and an overflow policy (backpressure or drop) applies at the cap, so the buffer self-trims and can never exhaust host memory or disk.
4. *No second configuration surface*: egress workers reconstruct each connector from the publisher definitions already present in the DAG, so operators configure destinations once.

The subsystem composes with multiprocess/worker execution to give *massively parallel* egress. Because a whole DAG is pinned to one worker process and DAG names are universally unique, each DAG’s publishers, its WALs, and its drainer co-locate in a single process; every worker drains its own WALs concurrently across all cores with no cross-process coordination on the hot path. The WAL is namespaced by the (universally unique) DAG name, so it follows the DAG and resumes correctly on whichever worker the DAG is scheduled to after a restart.

8. Related Work

8.1 Stream Processing Frameworks

Apache Flink [2] provides exactly-once stream processing with sophisticated windowing semantics. However, Flink’s Python API (PyFlink) introduces significant overhead due to cross-language serialization.

Apache Spark [3] pioneered in-memory distributed computing with the RDD abstraction. Spark Streaming processes data in micro-batches, introducing minimum latencies of 100ms-1s.

Naiad [4] introduced timely dataflow for iterative computations with low-latency incremental updates.

8.2 Zero-Copy Technologies

Apache Arrow [12] provides a columnar memory format enabling zero-copy data sharing. DishtaYantra extends this concept to arbitrary data structures via LMDB’s memory-mapped B-tree.

LMDB [8] is a high-performance key-value store used by DishtaYantra for zero-copy data exchange.

8.3 Workflow Orchestration

Prefect [13] and **Dagster** [14] are modern Python-native workflow orchestration tools, but lack DishtaYantra’s focus on microsecond-latency processing.

9. Experimental Evaluation

9.1 Experimental Setup

Hardware: - CPU: AMD Ryzen 9 5900X (12-core, 24-thread) @ 3.7GHz - RAM: 64GB DDR4-3200 - Storage: NVMe SSD (Samsung 980 PRO)

Software: - OS: Ubuntu 24.04 LTS - Python: 3.12.1 - Java: OpenJDK 21 - C++ Compiler: GCC 13.2 - Rust: 1.75.0

9.2 Workload Descriptions

We evaluate DishtaYantra using two representative workloads:

Workload	Description	Characteristics
W1: Passthrough	Data passes through DAG without computation	Measures framework overhead, I/O latency, serialization cost. Represents best-case latency and throughput achievable by the system.
W2: Mathematical Aggregation	Statistical calculations (mean, stddev, min, max, percentiles) on numerical arrays	CPU-bound workload testing calculator efficiency. Represents typical compute-bound analytics scenario.

9.3 Latency Results

Payload Size	P50	P95	P99	Max
1 KB	45 s	78 s	120 s	250 s
10 KB	65 s	110 s	180 s	350 s
100 KB	180 s	320 s	480 s	850 s
1 MB	850 s	1.2 ms	1.8 ms	3.2 ms

Table 1: End-to-end latency percentiles for W1 (Passthrough) workload

9.4 Throughput Results

Workers	W1 (Passthrough)	W2 (Math Aggregation)
1	180,000 msg/s	95,000 msg/s
2	350,000 msg/s	185,000 msg/s
4	680,000 msg/s	360,000 msg/s
6	1,000,000 msg/s	530,000 msg/s
8	1,320,000 msg/s	700,000 msg/s
10	1,620,000 msg/s	860,000 msg/s
12	1,950,000 msg/s	1,030,000 msg/s

Table 2: Maximum sustainable throughput (messages/second) by worker count

9.5 Scalability Analysis

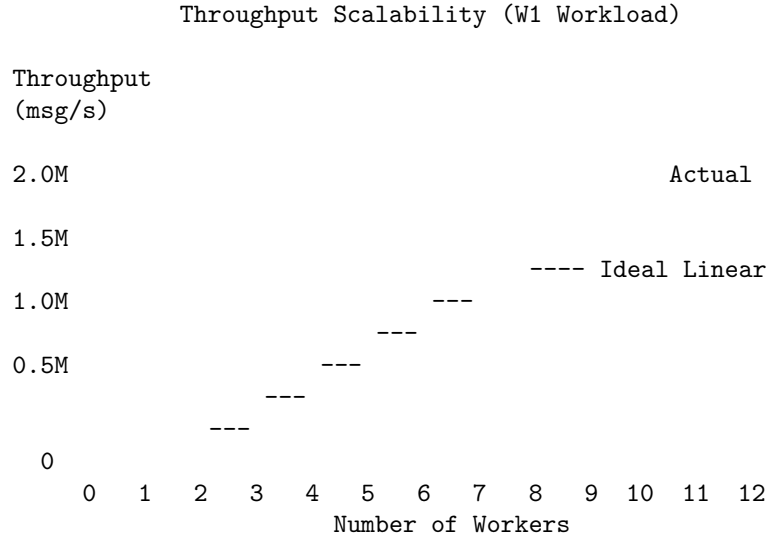


Figure 5: Throughput scalability showing near-linear scaling up to 12 workers

9.6 LMDB Zero-Copy Impact

Scenario	Without LMDB	With LMDB	Improvement
100KB to C++	5.2 ms	52 s	100×
1MB to Rust	48 ms	195 s	246×
10MB to Java	520 ms	2.1 ms	248×

Table 3: Impact of LMDB zero-copy on cross-language communication

10. Discussion

10.1 Design Trade-offs

Memory vs. Latency: LMDB’s memory-mapped design trades memory usage for latency. The default 1GB map size accommodates most workloads.

Process Isolation vs. Communication Overhead: The multiprocessing architecture eliminates GIL contention but introduces IPC overhead. LMDB pub/sub minimizes this cost.

Flexibility vs. Complexity: Supporting five languages increases implementation complexity but enables optimal algorithm selection.

10.2 Limitations

1. **Single-Node Focus:** Current implementation targets single-node deployments
2. **JVM Startup Time:** Java calculators require JVM initialization (~2-5 seconds)
3. **Memory Footprint:** Each worker process maintains independent memory

10.3 Future Work

1. **Distributed Mode:** Multi-node deployment with automatic DAG partitioning
 2. **GPU Calculators:** CUDA/ROCm integration for ML inference
 3. **Checkpoint/Recovery:** Exactly-once semantics with distributed checkpointing
 4. **WebAssembly Calculators:** Sandboxed calculator execution via WASM
-

11. Conclusion

We presented **DishtaYantra**, a high-performance multi-language dataflow framework that addresses critical limitations in existing systems. Through LMDB-based zero-copy data exchange, unified multi-language calculator support, and GIL-free worker parallelism, DishtaYantra achieves:

- **100-250× latency improvements** over traditional JSON serialization for large payloads
- **Sub-millisecond P99 latency** for payloads up to 100KB

- **90% parallel efficiency** at 12 workers on consumer hardware
- **Unified support** for Python, Java, C++, Rust, and REST calculators
- **Comprehensive pub/sub** integration with 10+ message brokers

DishtaYantra is open-source and available at <https://github.com/ajsinha/dishtayantra>.

References

- [1] T. Akidau et al., “MillWheel: Fault-tolerant stream processing at internet scale,” VLDB Endowment, 2013.
- [2] P. Carbone et al., “Apache Flink: Stream and batch processing in a single engine,” IEEE Data Engineering Bulletin, 2015.
- [3] M. Zaharia et al., “Apache Spark: A unified engine for big data processing,” Communications of the ACM, 2016.
- [4] D. G. Murray et al., “Naiad: A timely dataflow system,” SOSP, 2013.
- [5] M. Kleppmann, “Designing Data-Intensive Applications,” O’Reilly Media, 2017.
- [6] D. Beazley, “Understanding the Python GIL,” PyCon, 2010.
- [7] W. M. Johnston et al., “Advances in dataflow programming languages,” ACM Computing Surveys, 2004.
- [8] H. Chu, “LMDB: Lightning Memory-Mapped Database,” <https://www.symas.com/lmdb>, 2011.
- [9] W. Jakob et al., “pybind11,” <https://github.com/pybind/pybind11>, 2016.
- [10] PyO3 Contributors, “PyO3: Rust bindings for Python,” <https://pyo3.rs/>, 2017.
- [11] B. Dagenais, “Py4J,” <https://www.py4j.org/>, 2009.
- [12] Apache Software Foundation, “Apache Arrow,” <https://arrow.apache.org/>, 2016.
- [13] Prefect Technologies, “Prefect,” <https://www.prefect.io/>, 2018.
- [14] Elementl, “Dagster,” <https://dagster.io/>, 2019.

Disclaimer and Legal Notice

DISCLAIMER OF WARRANTIES

THE SOFTWARE IS PROVIDED “AS IS”, WITHOUT WARRANTY OF ANY KIND, EXPRESS OR IMPLIED, INCLUDING BUT NOT LIMITED TO

THE WARRANTIES OF MERCHANTABILITY, FITNESS FOR A PARTICULAR PURPOSE AND NONINFRINGEMENT. IN NO EVENT SHALL THE AUTHORS OR COPYRIGHT HOLDERS BE LIABLE FOR ANY CLAIM, DAMAGES OR OTHER LIABILITY, WHETHER IN AN ACTION OF CONTRACT, TORT OR OTHERWISE, ARISING FROM, OUT OF OR IN CONNECTION WITH THE SOFTWARE OR THE USE OR OTHER DEALINGS IN THE SOFTWARE.

LIMITATION OF LIABILITY

TO THE MAXIMUM EXTENT PERMITTED BY APPLICABLE LAW, IN NO EVENT SHALL THE AUTHOR, CONTRIBUTORS, OR COPYRIGHT HOLDERS BE LIABLE FOR ANY DIRECT, INDIRECT, INCIDENTAL, SPECIAL, EXEMPLARY, OR CONSEQUENTIAL DAMAGES (INCLUDING, BUT NOT LIMITED TO, PROCUREMENT OF SUBSTITUTE GOODS OR SERVICES; LOSS OF USE, DATA, OR PROFITS; OR BUSINESS INTERRUPTION) HOWEVER CAUSED AND ON ANY THEORY OF LIABILITY, WHETHER IN CONTRACT, STRICT LIABILITY, OR TORT (INCLUDING NEGLIGENCE OR OTHERWISE) ARISING IN ANY WAY OUT OF THE USE OF THIS SOFTWARE, EVEN IF ADVISED OF THE POSSIBILITY OF SUCH DAMAGE.

USAGE RESPONSIBILITY

Users of DishtaYantra are solely responsible for: - Ensuring compliance with all applicable laws and regulations in their jurisdiction - Validating the software's suitability for their specific use case - Implementing appropriate security measures for production deployments - Backing up data and maintaining disaster recovery procedures - Testing thoroughly in non-production environments before deployment

NO PROFESSIONAL ADVICE

This software and documentation do not constitute professional advice of any kind. Users should consult with qualified professionals for legal, financial, security, or other specialized guidance.

THIRD-PARTY COMPONENTS

DishtaYantra incorporates third-party open-source components, each subject to their respective licenses. Users are responsible for compliance with all applicable third-party licenses.

INDEMNIFICATION

By using this software, you agree to indemnify and hold harmless the author and contributors from any claims, damages, or expenses arising from your use of the software.

Contact: ajsinha@gmail.com

Repository: <https://github.com/ajsinha/dishtayantra>